



INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Arreglos en Java

Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

Notas de un parcial

n1	n2	n3	n4	n5	n6	n7	n8	n9	n10	n11	n12
90	70	40	15	60	75	100	100	30	70	70	70

Calcular el promedio de notas

$(n1 + n2 + n3 + n4 + n5 + n6 + n7 + n8 + n9 + n10 + n11 + n12) / 12$

Arreglos

Un arreglo es una estructura de datos **homogénea**, **lineal** y **ordenada** que permite representar un conjunto de valores.

90	70	40	15	60	75	100	100	30	70	70	70
----	----	----	----	----	----	-----	-----	----	----	----	----

- Todos sus elementos son del mismo tipo.
- Cada elemento tiene un predecesor, excepto el primero, y un sucesor, excepto el último.
- La posición de cada elemento establece una relación de orden.

Declaración de la variable

En el momento que se declara la variable se establece el nombre y el tipo de los elementos.

```
int [] notas;
```

90	70	40	15	60	75	100	100	30	70	70	70
----	----	----	----	----	----	-----	-----	----	----	----	----

En Java los corchetes se utilizan para declarar un arreglo

Creación del arreglo

En el momento que se crea el arreglo se establece la cantidad de elementos.

```
notas = new int[12];
```

90	70	40	15	60	75	100	100	30	70	70	70
----	----	----	----	----	----	-----	-----	----	----	----	----

En Java los corchetes se utilizan para crear un arreglo

Creación del arreglo

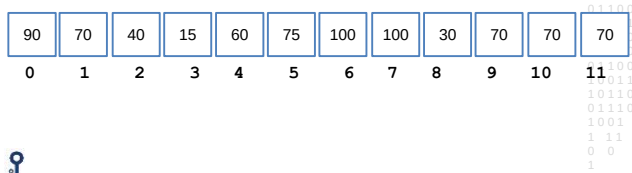
```
notas = new int[n];
```

90	70	40	15	60	75	100	100	30	70	70	70
----	----	----	----	----	----	-----	-----	----	----	----	----

En Java la cantidad de componentes puede establecerse con una expresión entera

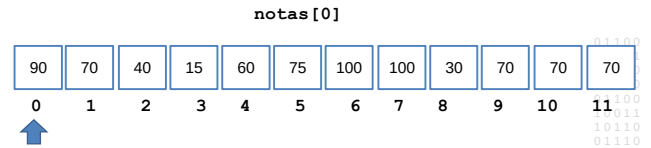
Índices

Cada elemento del arreglo se identifica y accede a través de un índice.



Índices

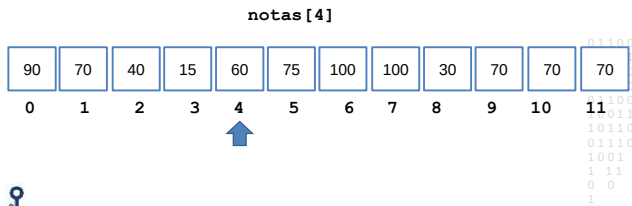
Cada elemento del arreglo se identifica y accede a través de un índice.



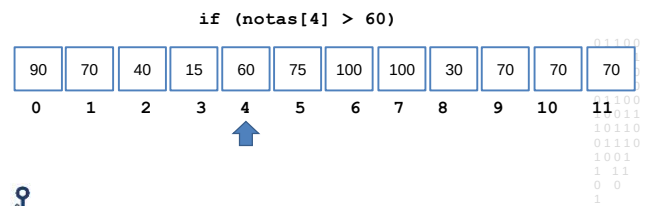
En Java el primer elemento de un arreglo se accede con el índice 0

Índices

Cada elemento del arreglo se identifica y accede a través de un índice.

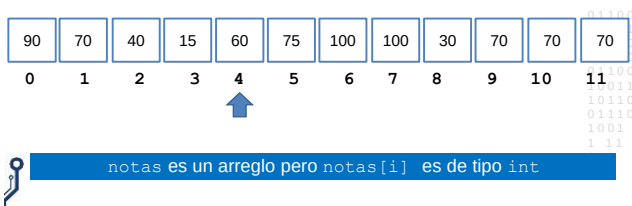


Expresiones con arreglos



Generalización

if (notas[i] > 60)



notas es un arreglo pero notas[i] es de tipo int

El atributo length

if (i < notas.length)
hoy = notas[i];

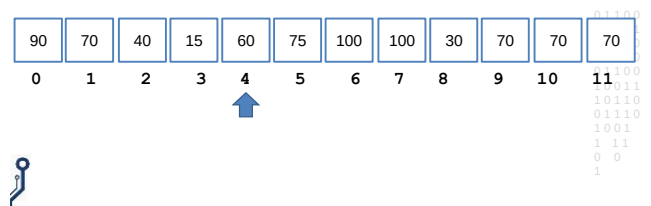
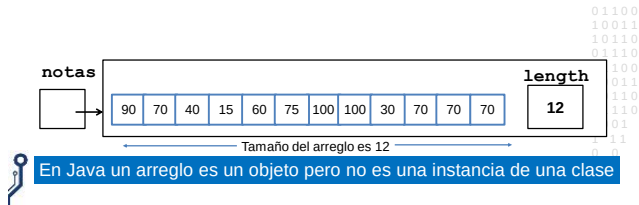
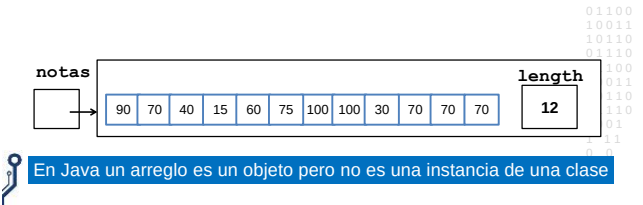


Diagrama de objetos



Recorridos sobre arreglos

- Calcular la cantidad de parciales aprobados.
- Calcular el promedio general.
- Calcular el promedio de los parciales aprobados.
- Decidir si hay al menos un parcial con nota 100.
- Decidir si hay exactamente un parcial con nota 100.



El diseño

NotasParcial	
<<atributos de instancia>> notas [] entero	
<<Constructores>> NotasParcial(cant: entero)	
<<Comandos>> establecerNota(n: entero, i:entero)	
<<Consultas>> obtenerNota(i: entero): entero cantNotas(): entero cantAprobados():entero promedioGeneral():real promedioAprobados():entero alMenos100():boolean exactamente100():boolean	• NotasParcial(cant: entero) Requiere cant mayor a 0 • establecerNota(n: entero, i:entero) Requiere i válido. Modifica el iésimo elemento si n está entre 0 y 100, en caso contrario no provoca cambios. • obtenerNota(i: entero): entero Requiere i válido. • cantNotas():entero Retorna el tamaño del arreglo
Requiere que todas las notas estén almacenadas antes de ejecutar una consulta	

El diseño

NotasParcial	
<<atributos de instancia>> notas [] entero	
<<Constructores>> NotasParcial(cant: entero)	
<<Comandos>> establecerNota(n: entero, i:entero)	
<<Consultas>> obtenerNota(i: entero): entero cantNotas(): entero cantAprobados():entero promedioGeneral():real promedioAprobados():real alMenos100():boolean exactamente100():boolean	• cantAprobados():entero Retorna la cantidad de parciales con nota mayor o igual a 60. • promedioGeneral():real Retorna el promedio de todos los parciales. • promedioAprobados():real Retorna el promedio de los parciales aprobados. Si no hay parciales aprobados retorna -1. • alMenos100():boolean Retorna true si hay al menos un parcial con nota 100. • exactamente100():boolean Retorna true si hay exactamente un parcial con nota 100.
Requiere que todas las notas estén almacenadas antes de ejecutar una consulta	

El tester

NotasParcial	
<<atributos de instancia>> notas [] entero	
<<Constructores>> NotasParcial(cant: entero)	
<<Comandos>> establecerNota(n: entero, i:entero)	
<<Consultas>> obtenerNota(i: entero): entero cantNotas(): entero cantAprobados():entero promedioGeneral():real promedioAprobados():real alMenos100():boolean exactamente100():boolean	Casos de prueba No hay ningún 100. Hay un solo 100 y es la primera nota. Hay un solo 100 y es la última nota. Hay un solo 100 y no es la primera nota ni la última. Todos los parciales obtuvieron 100. La primera y la segunda son 100. La primera y la última son 100.
Requiere que todas las notas estén almacenadas antes de ejecutar una consulta	• exactamente100():boolean Retorna true si hay exactamente un parcial con nota 100.

La implementación

NotasParcial	
<<atributos de instancia>> notas [] entero	
<<Constructores>> NotasParcial(cant: entero)	
<<Comandos>> establecerNota(n: entero, i:entero)	
<<Consultas>> obtenerNota(i: entero): entero cantNotas(): entero cantAprobados():entero promedioGeneral():real promedioAprobados():real alMenos100():boolean exactamente100():boolean	<pre> class NotasParcial{ //Atributos de instancia private int [] notas; //Constructor public NotasParcial (int cant){ notas = new int [cant]; } //Comandos public void establecerNota(int n, int i){ //Requiere i válido if (n >= 0 && n <=100) notas[i] = n; } //Consultas public int obtenerNota(int i){ //Requiere i válido return notas[i]; } public int cantNotas(){ return notas.length; } } </pre>
Requiere que todas las notas estén almacenadas antes de ejecutar una consulta	